Task 14 — Terraform Modules (EC2, ECR, S3)

1.Summary

This task refactors the flat Terraform configuration from Task 12 (EC2 + ECR) and Task 13 (EC2 + S3) into three reusable Terraform **modules** -`ec2`, `ecr`, and `s3` called from a single root configuration. The same infrastructure is described, but the code is now organized so each resource type is self contained, independently reusable, and callable with different inputs from any future root configuration.

Per the task's instructions, only `terraform init`, `terraform fmt`, `terraform validate` and `terraform plan` were ran.

2. Project Structure

```
task14-terraform-modules/
  modules/
    ec2/
      variables.tf   # instance inputs (AMI, type, subnet, SG rules,
etc.)
      main.tf         # security group + aws_instance resources
     outputs.tf      # instance_id, public_ip, public_dns, sg_id
    ecr/
      variables.tf   # repository name, scan/mutability settings
      main.tf         # aws_ecr_repository resource
      outputs.tf      # repository_url, repository_arn,
repository_name
    s3/
      variables.tf   # bucket name, versioning/encryption/public-
access flags
      main.tf         # bucket + versioning + encryption + public-
access-block
      outputs.tf      # bucket_name, bucket_arn, bucket_id
  versions.tf          # Terraform + AWS provider version constraints
```

```
  variables.tf          # root-level inputs, shared across all modules
  data.tf                # default VPC/subnet lookups
  main.tf                # calls module "ecr", module "s3", module
"ec2"
  outputs.tf             # surfaces each module's outputs
  user_data.sh.tpl       # EC2 bootstrap script (Docker + app +
S3/ECR env)
  terraform.tfvars       # deployment-specific values
```

3. Why Modules

Each module (`ec2`, `ecr`, `s3`) is a self contained unit with its own `variables.tf` (inputs) and `outputs.tf` (return values), and knows nothing about the other modules or the root configuration. The root `main.tf` is the only place that wires them together it calls each module with `source = "./modules/<name>"`, passes in the required variables, and threads outputs between them where needed (for example, the EC2 module's `user_data` script is rendered using `module.s3.bucket_name` and `module.ecr.repository_url`).

This means any of the three modules could be reused as is in a completely different root configuration for a different project, a different environment, or a different combination of resources without modification.

4. What Each Module Does

`modules/s3` — Creates the storage bucket and its three security configurations: versioning, AES-256 server-side encryption, and Block Public Access (all four settings). Takes `bucket_name` and toggles for each setting as inputs; returns the bucket's name, ARN, and ID.

`modules/ecr` — Creates a container registry repository for the Twenty CRM Docker image, with vulnerability scan-on-push enabled. Takes `repository_name` as input; returns the repository's pull/push URL and ARN.

`modules/ec2` — Creates a security group (SSH + configurable list of app ports) and the EC2 instance itself, attaching an existing IAM instance profile by name (no new IAM resources are created, consistent with Task 13's approach). Takes AMI, instance type,

subnet/VPC IDs, and a rendered `user_data` script as input; returns the instance ID, public IP, public DNS, and security group ID.

5. Root Configuration

`main.tf` at the root calls all three modules:

```
module "ecr" { source = "./modules/ecr" ... }
module "s3"  { source = "./modules/s3"  ... }
module "ec2" {
  source = "./modules/ec2"
  ...
  user_data = templatefile("${path.module}/user_data.sh.tpl", {
    s3_bucket_name     = module.s3.bucket_name
    ecr_repository_url = module.ecr.repository_url
    ...
  })
  depends_on = [module.s3, module.ecr]
}
```

`data.tf` looks up the existing default VPC/subnet (no new VPC created), and the root `variables.tf` holds every input shared across modules region, instance type, approved AMI list (enforced via validation blocks), bucket name, repository name, and so on populated at runtime from `terraform.tfvars`.

6. Commands Run

```
terraform init
terraform fmt
terraform validate
terraform plan
```

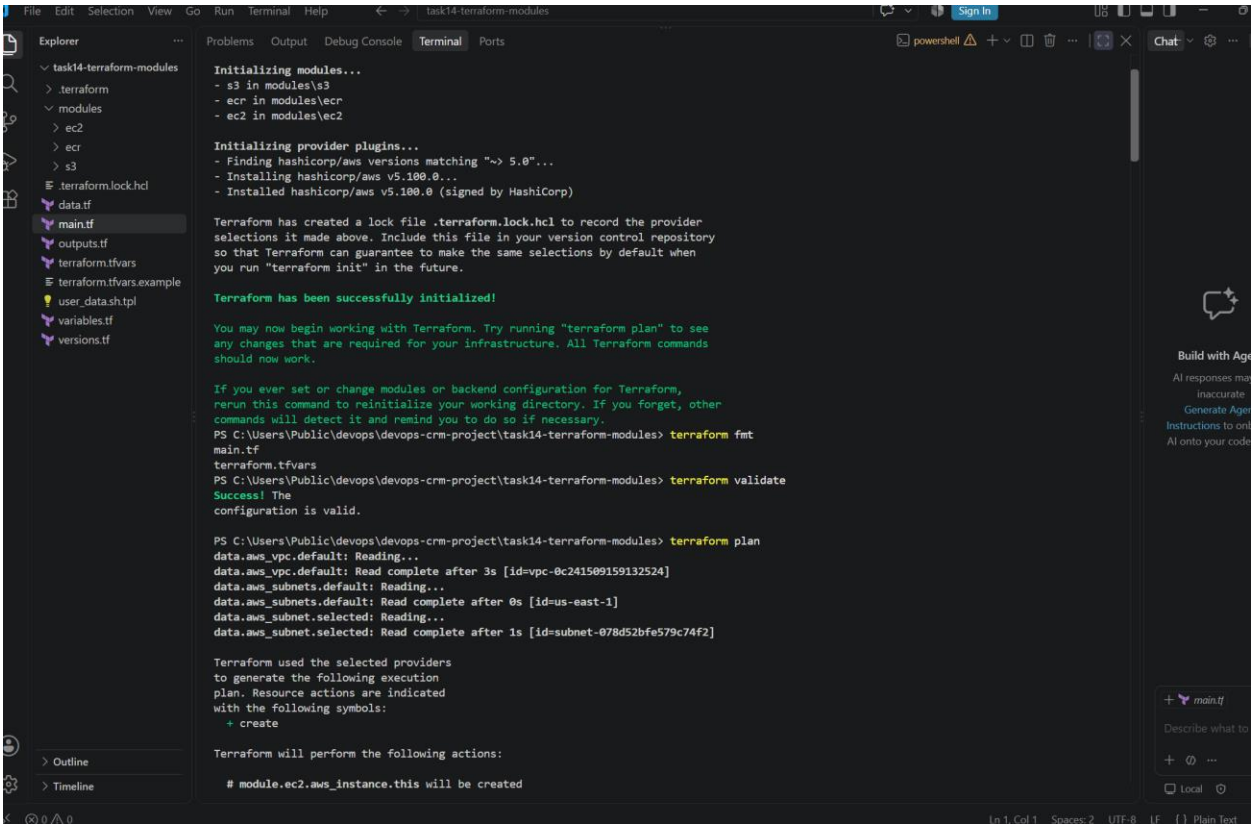
6.1 `terraform init`

Successfully initialized all three modules and the AWS provider:

```
Initializing modules...
- s3 in modules\s3
- ecr in modules\ecr
- ec2 in modules\ec2

Initializing provider plugins...
- Installing hashicorp/aws v5.100.0...

Terraform has been successfully initialized
```



6.2 `terraform fmt`

Reformatted `main.tf` and `terraform.tfvars` to canonical style confirms the configuration follows standard formatting conventions.

6.3 `terraform validate`

```
Success! The configuration is valid.
```

Confirms all module calls, variable references, and resource blocks are syntactically correct and internally consistent.

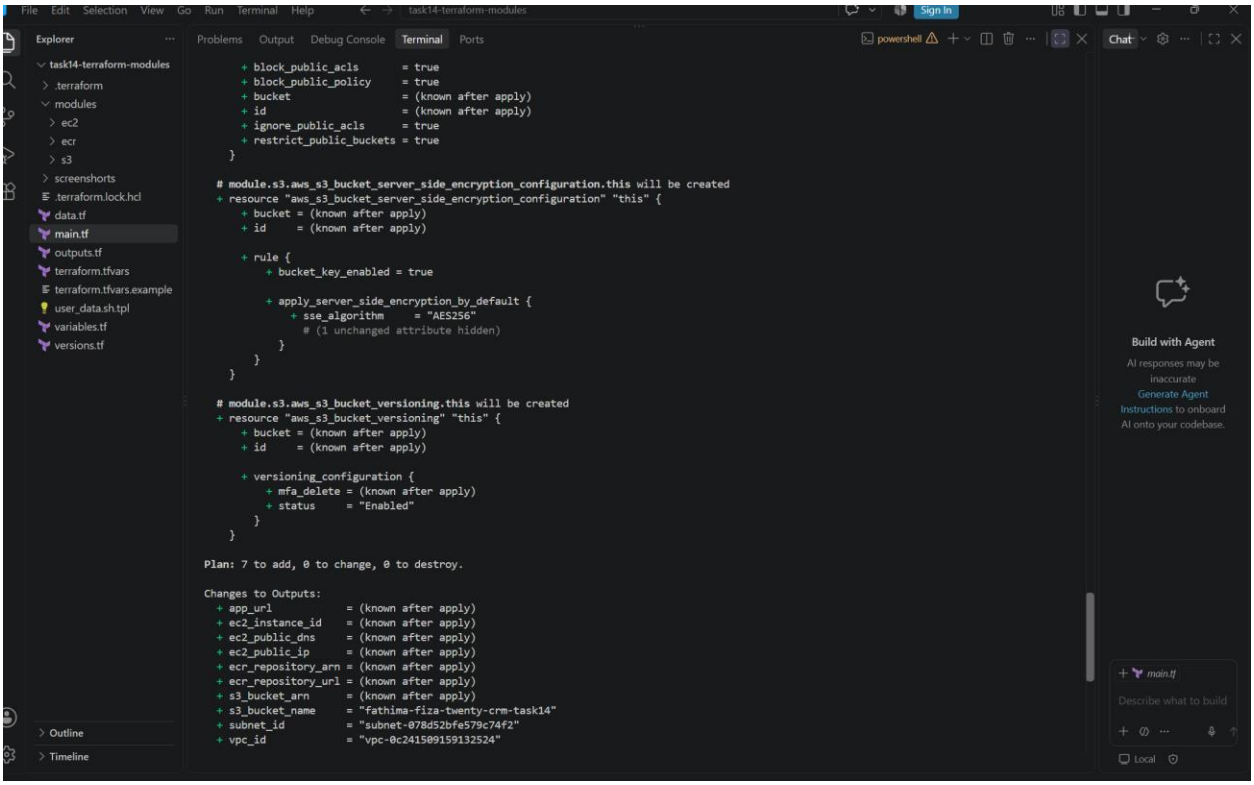
6.4 `terraform plan`

Ran against the live default VPC/subnet data sources successfully read `vpc-0c241509159132524` and `subnet-078d52bfe579c74f2` — then produced a full plan proposing to create every resource fresh (S3 bucket + its 3 config resources, ECR repository, EC2 instance + security group):

```
Plan: 7 to add, 0 to change, 0 to destroy.
```

Reviewed resource-by-resource in the output for example, the S3 module's `aws_s3_bucket_public_access_block` proposal shows all four settings (`block_public_acls`, `block_public_policy`, `ignore_public_acls`, `restrict_public_buckets`) set to `true`, and `aws_s3_bucket_versioning` shows `status = "Enabled"` — confirming the S3 module correctly enforces the same security requirements as Task 13, just now sourced from a reusable module instead of inline resources.

The `Changes to Outputs` section confirms the root `outputs.tf` correctly surfaces values from every module: `app_url`, `ec2_instance_id`, `ec2_public_dns`, `ec2_public_ip`, `ecr_repository_arn`, `ecr_repository_url`, `s3_bucket_arn`, `s3_bucket_name`, `subnet_id`, `vpc_id`.


File Edit Selection View Go Run Terminal Help
task14-terraform-modules
Sign In
Explorer
task14-terraform-modules
.terraform
modules
ec2
ecr
s3
screenshorts
.terraform.lock.hcl
data.tf
main.tf
outputs.tf
terraform.tfvars
terraform.tfvars.example
user_data.sh.tpl
variables.tf
versions.tf
Outline
Timeline
Problems Output Debug Console Terminal Ports
powershell
+ block_public_acls = true
+ block_public_policy = true
+ bucket = (known after apply)
+ id = (known after apply)
+ ignore_public_acls = true
+ restrict_public_buckets = true
}
# module.s3.aws_s3_bucket_server_side_encryption_configuration.this will be created
+ resource "aws_s3_bucket_server_side_encryption_configuration" "this" {
+ bucket = (known after apply)
+ id = (known after apply)
+ rule {
+ bucket_key_enabled = true
+ apply_server_side_encryption_by_default {
+ sse_algorithm = "AES256"
# (1 unchanged attribute hidden)
}
}
}
# module.s3.aws_s3_bucket_versioning.this will be created
+ resource "aws_s3_bucket_versioning" "this" {
+ bucket = (known after apply)
+ id = (known after apply)
+ versioning_configuration {
+ mfa_delete = (known after apply)
+ status = "Enabled"
}
}
Plan: 7 to add, 0 to change, 0 to destroy.
Changes to Outputs:
+ app_url = (known after apply)
+ ec2_instance_id = (known after apply)
+ ec2_public_dns = (known after apply)
+ ec2_public_ip = (known after apply)
+ ecr_repository_arn = (known after apply)
+ ecr_repository_url = (known after apply)
+ s3_bucket_arn = (known after apply)
+ s3_bucket_name = "fathima-fiza-twenty-crm-task14"
+ subnet_id = "subnet-078d52bfe579c74f2"
+ vpc_id = "vpc-0c241509159132524"
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
main.tf
Describe what to build
Local